研究方案：未说完就开始翻译

基于并行未来分支推测的 LLM 流式同声传译

> 大数据并行计算 2026 秋季学期 期末项目
> 团队：待填（2–5 人）
> 日期：2026-11-01

摘要

同传最根本的延迟不是"算得慢"，而是**必须等一句话说完才能确定该怎么翻**。词序差异（如德语、日语的动词在句末）和句法歧义使得目标词依赖于尚未到达的源词；如果等整句输入再翻译，延迟至少是一句话的时长加上翻译时间。

本项目的主线是：**不等未来确定，而是并行地把"可能的未来"先翻译掉**。具体提出 **PFBA（Parallel Future-Branch Anticipation，并行未来分支推测）**：在当前源前缀 $x_{\le i}$ 上采样 B 个可能的未来延续，并行翻译每个"前缀 + 未来"分支，提交所有分支翻译的**一致前缀**；当真实未来到达时，用验证结果保留匹配分支、丢弃其余分支。并行算力用来覆盖未来的不确定性；KV 复用、批处理只是支撑手段，不是主线。

核心问题从"如何加速已有计算"变成"**如何用并行计算把未来的不确定性提前消化**"。核心指标：词序等待时间、提交一致前缀的长度、分支覆盖率、验证接受率、墙钟 lag 的 p50/p95、以及相对 oracle（已知真实未来）的可预测性差距。

0. 范围声明：主线与支撑

主线（科学问题）：在源句未完成、未来未知时，如何决定"现在翻译什么、提交什么"，从而把"等整句"的延迟转化为"并行推测未来"的延迟。

支撑手段（工程问题）：KV/prefix 复用、连续批处理、树形 attention、TP。它们降低分支翻译的成本，让 B 条分支能塞进实时预算，但不是本项目的贡献主张。

本轮不做：多策略 portfolio、prefix-tree 训练、commit head、ASR/TTS 细节、投机解码的 target 侧变体。它们要么是质量问题，要么是训练成本问题，不是"未说完就开始翻译"这一根本延迟问题。

1. 问题分析：为什么"等一句话说完"是根本延迟

1.1 延迟的真正来源

朴素系统的延迟是：

$$T_{\text{total}} = T_{\text{wait}} + T_{\text{translate}}$$

一句 10 秒的话，等说完再翻，延迟至少 10 秒；而人做同传时，听到前几个词就开始输出。**等待本身才是主项**，翻译计算只是叠加在后面的第二项。

1.2 词序延迟：目标词依赖尚未到达的源词

设目标词 y_j 主要对齐到源位置 $\alpha(j)$。在不推测未来时，y_j 最早只能在 $x_{\alpha(j)}$ 到达后产生：

$$\tau_j \geq a_{\alpha(j)} + c_{\text{dec}}$$

因此逐词延迟满足：

$$\text{lag}_j = \tau_j - a_j \geq \underbrace{a_{\alpha(j)} - a_j}_{\text{词序等待}} + \underbrace{c_{\text{dec}}}_{\text{计算}}$$

对 SOV→SVO（德/日→英/中），动词在句末，$\alpha(j) \gg j$，**词序等待**是延迟主项。例：

```
德语: Ich will, dass du nach oben gehst
英语: I want you to go upstairs
```

"gehst" 在句末，但 "go" 在英语里必须早出现。等 "gehst" 到达再翻，等于等整句；提前翻又必须"猜"这个动词。**这就是同传的核心矛盾。**

1.3 歧义延迟：前缀的译文取决于后续内容

即使没有词序问题，前缀本身也可能有多个解释：

```
The president of the company | which was founded in 1990 | announced ...
```

关系从句修饰谁、介词短语挂在哪里、某个词取哪个义项，都要等后续内容才能确定。中文里定语必须前置，歧义会被放大：早翻可能翻错，晚翻则延迟。

1.4 不可逆性：提交后不能改

同传的输出一旦提交就不能修改（flicker-free）。因此"早翻"不仅是准确率问题，还是**风险**问题：猜错后无法回退。re-translation 允许修改，但会造成字幕闪烁和重复计算；很多场景不接受。

1.5 现有方法的天花板

方法	做法	根本局限
wait-k	先读 k 个词，之后读 1 写 1	固定等待，不区分"可预测"与"不可预测"
prefix-to-prefix	用前缀对训练，隐式 anticipation	仍是单条未来，不显式枚举
stable prefix / chunk	稳定了才提交	稳定之前只能等
re-translation	先翻，后到再改	闪烁、重复计算
adaptive policy	学习 READ/WRITE	只决定"等不等"，不决定"猜什么"

它们共同的前提是：**未来是未知的，所以只能等或赌**。没有人把"未来"显式地展开成多个分支。

1.6 关键洞察

未来虽然未知，但**往往可预测**：日常语句、会议套话、ASR 的 n-best、上下文的强约束，都让"下一小段源文"有很高的条件概率。于是可以把问题改写为：

> 不是"等未来确定再翻"，而是"**把可能的未来并行翻掉，只提交对所有未来都成立的部分**"。

这把"预测未来"从**风险**变成了**可并行计算的资源**：分支越多，覆盖越高；一致前缀越长，能提前提交的越多；真实未来到达时，验证只需做一次匹配。

2. 新方案：PFBA（并行未来分支推测）

2.1 核心思想

在当前前缀 $x_{\leq i}$ 上：

1. **采样未来**：用预测器 q_ϕ 生成 B 个可能的源延续 $\hat{x}^{(b)}_{i+1:i+h}$；
2. **并行翻译**：对每个分支翻译 $x_{\le i} + \hat{x}^{(b)}$，得到 $y^{(b)}$；
3. **一致提交**：取所有 $y^{(b)}$ 的最长公共前缀 $\bar{y}$，提交 $\bar{y}$；
4. **到达验证**：真实未来 $x_{i+1:i+\delta}$ 到达后，找匹配的分支 b^*，保留其草稿；无匹配则回退。

2.2 形式化

未来采样：

$$\hat{x}^{(b)}_{i+1:i+h} \sim q_\phi\left(\cdot \mid x_{\le i}\right), \qquad b = 1, \dots, B$$

分支翻译：

$$y^{(b)} \sim p_\theta\left(\cdot \mid x_{\le i},\, \hat{x}^{(b)}_{i+1:i+h}\right)$$

一致前缀：

$$\bar{y} = \mathrm{LCP}\left(y^{(1)}, \dots, y^{(B)}\right)$$

提交 $\bar{y}$，其余 $y^{(b)}$ 作为草稿保留。真实未来到达后：

$$b^* = \arg\min_b \; d\left(\hat{x}^{(b)}_{i+1:i+\delta},\, x_{i+1:i+\delta}\right)$$

若 $d\big(\hat{x}^{(b^*)}, x_{i+1:i+\delta}\big) \le \epsilon$，从 $y^{(b^*)}$ 继续；否则回退到"用真实前缀重新翻译"。

2.3 为什么能降低延迟

- **词序延迟**：如果分支覆盖了真实未来，句末动词对应的目标词已经提前翻好，真实动词到达时只需验证，不再等待翻译；
- **歧义延迟**：一致前缀只包含"对所有采样未来都成立"的部分，歧义部分不提交，等真实未来到达后再决定；
- **不可逆性**：提交的是跨未来一致的部分，天然 flicker-free；不一致的部分留在草稿里，不对外输出；
- **残余延迟**：只由"真实未来不在分支集合中"的部分决定，这是**不可预测的信息延迟**，也是理论下界。

2.4 与 speculative decoding 的类比

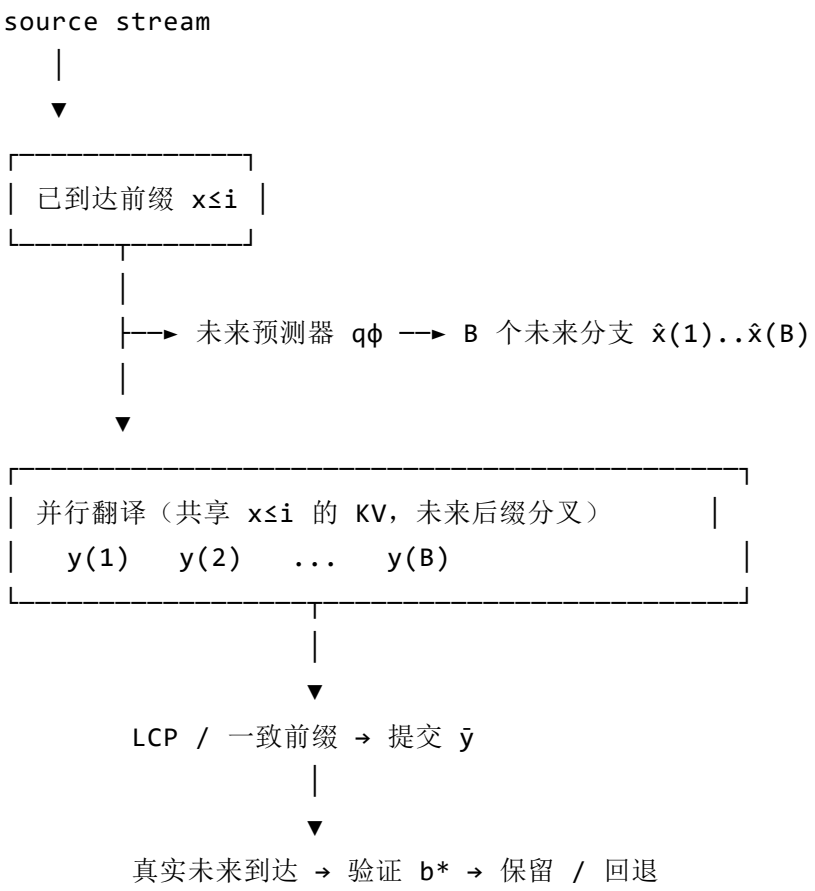
	speculative decoding	PFBA
推测对象	目标 token	**源未来**
草稿来源	draft model	未来预测器 q_ϕ
验证依据	主模型概率	**真实到达的源文**
并行单元	多个候选 token	多个未来分支
失败代价	丢弃草稿	丢弃分支、回退翻译

一句话：**PFBA 是把 speculative execution 从"猜输出"搬到"猜输入"**。

2.5 并行结构

- **分支并行**：B 个未来分支相互独立，可批量解码；
- **前缀共享**：所有分支共享 $x_{\le i}$ 的 KV，只在未来后缀处分叉（tree attention / prefix cache）；
- **验证归约**：真实未来到达后，B 个分支做一次匹配归约，选出 b^*；
- **提交串行**：一致前缀的提交是串行段，受 Amdahl 限制；
- **预算调度**：在 $\rho < 1$ 下选择 B 和预测长度 h。

2.6 系统结构



2.7 设计选择

未来预测器：

- ASR n-best / lattice：真实语音系统里天然存在的未来不确定性，最便宜；
- n-gram / kenlm：文本场景的轻量基线；
- 小模型采样：质量更高，成本更高；
- oracle：直接用真实未来，作为上界。

预测长度 h：短（1–3 词）覆盖率高、分支差异小；长（整句）覆盖难、但词序收益大。需要实验找拐点。

一致规则：严格 LCP；或放宽的 RALCP（允许编辑距离 $\leq \epsilon$）；或置信度加权。

验证：精确匹配、编辑距离、embedding 相似度；阈值决定"接受草稿"还是"回退"。

回退：wait-k / stable-prefix / 重新翻译；回退频率决定浪费的算力。

2.8 一致提交的安全性

一致前缀只对**采样到的未来**成立，不保证对真实未来成立。安全提交需要：

$$\Pr_{z\sim q_\phi}\left(\bar{y} \preceq f_\theta(x_{\le i}, z)\right) \ge 1-\delta$$

即"随机未来下 $\bar{y}$ 仍是译文前缀"的概率足够高。实践上：

- 用更分散的未来集合（高温采样、ASR lattice）提高覆盖；
- 对低置信度片段不提交，等真实未来；
- 用验证结果在线校准阈值，控制 flicker 率。

3. 理论分析

3.1 覆盖率

定义 B 个分支、预测长度 h 的未来覆盖率：

$$\mathrm{Cov}(B,h) = \Pr\left(\exists b \in [B]:\ \hat{x}^{(b)}_{i+1:i+h} = x_{i+1:i+h}\right)$$

允许模糊匹配时：

$$\mathrm{Cov}_\epsilon(B,h) = \Pr\left(\min_b d\left(\hat{x}^{(b)}_{i+1:i+h},\, x_{i+1:i+h}\right) \le \epsilon\right)$$

覆盖率越高，能提前提交的目标词越多。

3.2 期望延迟

$$\mathbb{E}[\mathrm{lag}] = \mathrm{Cov}\cdot\mathbb{E}\left[\mathrm{lag} \mid \mathrm{covered}\right] + (1-\mathrm{Cov})\cdot\mathbb{E}\left[\mathrm{lag} \mid \mathrm{not\ covered}\right]$$

覆盖时，词序等待被"预测 + 预翻译"吸收；未覆盖时，退化为原来的等待。**并行不能消除第二项**，只能通过提高覆盖率降低它的权重。

3.3 一致长度与分支数的权衡

$$L_{\mathrm{cons}}(B) = \left|\mathrm{LCP}\left(y^{(1)}, \ldots, y^{(B)}\right)\right|$$

- B 增大：覆盖率上升，但分支更分散，一致前缀变短，计算成本上升；

- B 减小：一致前缀更长，但覆盖不足，猜错风险上升。

因此存在一个"有效提前量"：

$$G(B) = \mathrm{Cov}(B) \cdot L_{\mathrm{cons}}(B)$$

$B^* = \arg\max_B G(B)$ 是实验要测的核心曲线。

3.4 计算预算

设单分支翻译成本 c_{branch}、合并成本 c_{merge}、源到达周期 Δ：

$$\rho(B) = \frac{B\, c_{\mathrm{branch}} + c_{\mathrm{merge}}}{\Delta}$$

实时条件 $\rho(B) < 1$，于是：

$$B_{\max} = \left\lfloor \frac{\Delta - c_{\mathrm{merge}}}{c_{\mathrm{branch}}} \right\rfloor$$

这就是"并行推测"的预算上限：分支不是越多越好，而是要在 Δ 内跑完。

3.5 Oracle 上界与可预测性差距

- **Oracle**：令 $\hat{x} = x_{>i}$（直接给真实未来），测"如果未来已知，延迟能降到多少"；
- **潜在收益**：$G_{\mathrm{potential}} = T^{\text{no-antic}} - T^{\mathrm{oracle}}$；
- **实际收益**：$G_{\mathrm{realized}} = T^{\text{no-antic}} - T^{\mathrm{PFBA}}$；
- **可预测性差距**：$G_{\mathrm{predictability}} = T^{\mathrm{PFBA}} - T^{\mathrm{oracle}}$。

三者把"未来不可知"造成的延迟拆成"可预测但没预测到"与"本质上不可预测"两部分。这是本项目最有价值的分析。

4. 研究问题与假设

RQ1（可预测性）：源未来有多可预测？覆盖率随 B、h、预测器（n-gram / 小模型 / ASR n-best / oracle）如何变化？

RQ2（延迟收益）：PFBA 相对 wait-k、stable-prefix、re-translation、no-anticipation，能否在相同质量下显著降低词序等待与墙钟 lag？

RQ3（一致性与安全）：一致前缀长度与 B 如何权衡？提交的 flicker 率能否控制在可接受范围？

RQ4（并行预算）：在 $\rho(B) < 1$ 下，最优 B 是多少？并行推测能否"免费"利用源到达的空闲算力？

H1：真实未来在 $B = 8$–16、短预测长度下覆盖率较高（尤其 ASR n-best / 会议套话）。

H2：PFBA 在 SOV→SVO（de→en、ja→zh）上的延迟收益显著大于 SVO→SVO（en→zh），因为词序等待更大。

H3：一致前缀提交的 flicker 率随 B 增大而下降（覆盖更全），但一致长度也下降；存在最优 B。

H4：当 $\rho(B) < 1$ 时，增加 B 几乎不增加墙钟延迟，却提高覆盖率；当 $\rho(B) \geq 1$ 时，延迟随 B 线性上升。

5. 实验设计

5.1 数据与模型

- 数据：MUST-C（de→en，词序差异大）、CoVoST2（en→zh / zh→en）、FLEURS；文本转写模拟流式，chunk $= 0.5/1/2$ s。
- 模型：Qwen2.5-7B-Instruct、Qwen3-8B；系统实验用 1.5B/3B。
- 未来预测器：kenlm n-gram、Qwen-0.5B 采样、ASR n-best（可选）、oracle。
- 硬件：1–4× 24–80GB GPU；记录 CPU/内存/NVLink/PCIe/驱动/CUDA/PyTorch 版本。

5.2 基线

- **等整句**（full-sentence）：质量上界、延迟下界最差；
- **wait-k**：$k = 1, 3, 5, 7, 9$；
- **stable-prefix / chunk-based**：稳定了才提交；
- **re-translation**：先翻后改；
- **no-anticipation**：只翻已到达前缀，不预测未来；
- **oracle future**：直接给真实未来，测上界；
- **PFBA**：$B = 2, 4, 8, 16$，$h = 1, 2, 4$ 词。

5.3 指标

- **延迟**：词序等待 $a_{\alpha(j)} - a_j$、墙钟 lag p50/p95/p99、TTFT、提交提前量；
- **推测质量**：覆盖率 $\mathrm{Cov}(B, h)$、验证接受率、一致前缀长度 L_{cons}、flicker 率；
- **翻译质量**：BLEU / chrF / COMET（iso-latency 比较）；
- **并行效率**：$\rho(B)$、$B_{\max}$、tokens/s、GPU 利用率、加速比；

- **分析**：$G_{\text{potential}}$、G_{realized}、$G_{\text{predictability}}$。

5.4 实验矩阵

编号	实验	变量	目的
E1	词序延迟分析	语言对、句子长度	量化 $a_{\alpha(j)} - a_j$ 的分布， 找最需要推测的片段
E2	Oracle 上界	真实未来 vs 无推测	测"未来已知能省多少"
E3	覆盖率	B、h、预测器	可预测性曲线
E4	PFBA vs 基线	wait-k / stable / re-translation	iso-latency BLEU、lag
E5	一致性与安全	LCP vs RALCP、阈值	一致长度、flicker 率
E6	并行预算	B、batch、GPU 数	$\rho(B)$、$B_{\max}$、scaling
E7	语言对差异	de→en、ja→zh、en→zh	验证 H2

5.5 评测协议

- 固定数据划分，不在 test 上调参；
- 同一质量策略、同一解码参数下比较；
- warmup 20%，重复 5 次，median/IQR；延迟报 p50/p95/p99；
- 用同一词对齐（fast_align / awesome-align）计算 $\alpha(j)$ 与 lag；
- 记录硬件/软件版本；代码、配置、日志、CSV、绘图脚本入库；
- README 说明环境、数据、运行命令。

5.6 预期结果

- E1：de→en 的词序等待显著大于 en→zh，动词/从句是主要等待来源；
- E2：oracle 能把词序等待降低 40%–70%，说明"未来已知"有很大空间；
- E3：短预测长度下 $B = 8$–16 覆盖率较高；长预测长度覆盖率快速下降；
- E4：PFBA 在 iso-latency 下 BLEU 提升，或在 iso-BLEU 下 lag 降低；
- E5：B 增大时 flicker 率下降、一致长度下降，存在最优 B；
- E6：$\rho < 1$ 时增加 B 几乎不增加延迟；$\rho \geq 1$ 时延迟线性上升。

若覆盖率低、收益不显著，也构成有价值的负结果：说明该语言对/场景的信息延迟本质上不可预测，需要靠"等"或"改"而不是"猜"。

6. 预期成果与贡献

交付物：

- PFBA 原型：未来预测器 + 并行分支翻译 + 一致提交 + 到达验证；
- 词序延迟与可预测性分析；
- 覆盖率/一致长度/延迟/质量曲线；
- 代码 + README + ≤ 5 页 IEEE 报告 + 海报。

贡献：

1. 把"未说完就开始翻译"形式化为**未来分支推测**问题，给出词序等待的延迟分解；
2. 提出 PFBA：并行翻译多个未来分支、提交跨未来一致前缀、到达后验证，兼顾低延迟与 flicker-free；
3. 用 oracle 上界与可预测性差距，量化"并行推测到底能消除多少等待"，并给出 $\rho(B) < 1$ 下的最优 B。

7. 风险与备选

风险	概率	影响	缓解	备选
未来预测器覆盖率低	高	高	先用 oracle 测上界；再用 ASR n-best / 短 h	转为"可预测性上界"分析
一致前缀太短	中	中	放宽 RALCP、置信度加权	退回 stable-prefix
提交不安全（flicker）	中	高	保守阈值、低置信不提交	只提交验证后的部分
分支计算太贵	中	中	$B_{\max}$、批处理、小模型	减 B、缩 h
实现复杂	中	高	先 oracle + n-gram 文本版	砍 ASR/tree attention
延迟测量噪声	中	中	warmup/repeats/隔离	报告 CI

MVP：文本版，oracle 未来 + kenlm，$B = 4$，de→en 与 en→zh，跑 E1/E2/E3/E4。仅此就能回答"等待能省多少、能预测多少"。

Stretch：小模型预测器、ASR n-best、tree attention、B 自适应、RALCP。

8. 分工与进度

8.1 分工（5 人）

角色	负责	交付
A 数据与对齐	数据、词对齐、lag 指标、复现	数据管线、E1
B 未来预测器	n-gram / 小模型 / ASR n-best / oracle	覆盖率、E3
C 分支翻译与服务	共享前缀 KV、批处理、tree attention	PFBA 推理栈、E6
D 一致性与调度	LCP/RALCP、验证、阈值、$\rho(B)$	E4/E5、B^*
E 集成与展示	集成、实验管理、报告/海报	demo、报告

若 4 人：A 兼 E；若 3 人：A 兼 E，B 兼 D。

8.2 进度（7 周，至 2026-12-21）

周	时间	任务	里程碑
W1	11/01–11/07	提案、数据、词对齐、wait-k/等整句基线	提案定稿
W2	11/08–11/14	词序延迟分析、oracle 上界	E1/E2
W3	11/15–11/21	kenlm 预测器、覆盖率	E3
W4	11/22–11/28	PFBA 分支翻译 + 一致提交 + 验证	E4
W5	11/29–12/05	一致长度/flicker、B 扫描	E5
W6	12/06–12/12	并行预算/扩展、语言对、补实验	E6/E7
W7	12/13–12/19	报告、海报、代码/README	提交
—	12/21	课堂展示	Poster + demo

9. 参考文献

1. Ma et al. STACL: Simultaneous Translation with Implicit Anticipation and Controllable Latency using Prefix-to-Prefix Framework. ACL 2019. 链接
2. Gu, Im, Li. Learning to Translate in Real-time with Neural Machine Translation. EACL 2017. 链接
3. Arivazhagan et al. Monotonic Infinite Lookback Attention for Simultaneous Machine Translation. ACL 2019. 链接
4. Liu, Spanakis, Niehues. Low-Latency Sequence-to-Sequence Speech Recognition and Translation by Partial Hypothesis Selection. arXiv:2005.11185, 2020. 链接
5. Arivazhagan et al. Re-translation versus Streaming for Simultaneous Translation. IWSLT 2020. 链接
6. Polák et al. CUNI-KIT System for Simultaneous Speech Translation Task at IWSLT 2022. IWSLT 2022. 链接
7. Agostinelli et al. Simul-LLM: A Framework for Exploring High-Quality Simultaneous Translation with Large Language Models. ACL 2024. 链接
8. Koshkin, Sudoh, Nakamura. TransLLaMa: LLM-based Simultaneous Translation System. EMNLP Findings 2024. 链接
9. Wang et al. Simultaneous Machine Translation with Large Language Models. ALTA 2024. 链接
10. Hoang, Axelrod, Post. Dynamic Lagging using Stable-Prefix Training for Simultaneous Translation. WMT 2026. 链接
11. Leviathan, Kalman, Matias. Fast Inference from Transformers via Speculative Decoding. ICML 2023. 链接
12. Chen et al. Accelerating Large Language Model Decoding with Speculative Sampling. arXiv:2302.01318, 2023. 链接
13. Kwon et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. SOSP 2023. 链接
14. Dao et al. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. NeurIPS 2022. 链接
15. Ma et al. SimulEval: A Toolkit for Simultaneous Translation. EMNLP 2020 (demo). 链接
16. Di Gangi et al. MuST-C: a Multilingual Speech Translation Corpus. NAACL 2019. 链接
17. Wang et al. CoVoST 2 and Massively Multilingual Speech Translation. Interspeech 2021. 链接

附录 A：指标定义

词序等待：

$$W_j = a_{\alpha(j)} - a_j$$

逐词 lag：

$$\mathrm{lag}_j = \tau_j - a_j$$

平均滞后：

$$\mathrm{AL} = \frac{1}{M} \sum_{j=1}^{M} \mathrm{lag}_j$$

覆盖率：

$$\mathrm{Cov}(B, h) = \Pr\left(\exists b:\ \hat{x}^{(b)}_{i+1:i+h} = x_{i+1:i+h}\right)$$

一致长度：

$$L_{\mathrm{cons}}(B) = \left|\mathrm{LCP}\left(y^{(1)}, \ldots, y^{(B)}\right)\right|$$

有效提前量：

$$G(B) = \mathrm{Cov}(B) \cdot L_{\mathrm{cons}}(B)$$

计算占空比与分支上限：

$$\rho(B) = \frac{B\, c_{\mathrm{branch}} + c_{\mathrm{merge}}}{\Delta}, \qquad B_{\max} = \left\lfloor \frac{\Delta - c_{\mathrm{merge}}}{c_{\mathrm{branch}}} \right\rfloor$$

附录 B：核心伪代码

```
for each chunk arrival at prefix i:
    # 1. 并行采样未来
    futures = predictor.sample(x[:i], B, h)
    # 2. 并行翻译（共享前缀 KV）
    drafts = batch_translate(x[:i], futures)
    # 3. 一致提交
    commit(LCP(drafts))
    # 4. 真实未来到达后验证
    b = argmin distance(futures, actual_future)
    if distance(b) <= eps:
        keep(drafts[b])
    else:
        fallback_translate(x[:i+delta])
```

附录 C：实验配置模板

```yaml
model: Qwen2.5-7B-Instruct
data: {test: MUST-C-t1, pair: de-en, chunk_ms: 1000}
predictor: {type: kenlm, B: 4, h: 2}
baselines: [full_sentence, wait_k, stable_prefix, re_translation, no_anticipation, oracle
metrics: [word_order_wait, coverage, acceptance, lcp_len, flicker, p50_lag, p95_lag, bleu
protocol: {warmup: 0.2, repeats: 5}
```